

LocalBuka Intern Case Study

Track: Frontend Engineering

Time Limit:	3-5 Days (Take-Home)
Starter Repo:	https://github.com/TobiAkinkuotu/assessment-repo
Submission:	Email your public fork link & 2-min Loom to hellohr@localbuka.com

Context Overview

In this frontend track, you will build a clean React web application with TypeScript, implementing discovery search dashboard and optimistic state updates matching LocalBuka's branding aesthetic.

Step-by-Step Engineering Flow

Phase 1: Layout Structure & Design System

Initialize your styling architecture using Vanilla CSS or TailwindCSS. Construct a layout consisting of a Navigation Header and search filters. Avoid default browser elements. Focus on responsive padding and margins for layout container.

Phase 2: Restaurant Discovery Feed & Cards

Implement a list display of restaurant profiles. Integrate dynamic card elements showing name, cuisine, rating stars, and tags.

- Render custom skeleton UI loading cards when fetching state is active.
- Handle image load errors cleanly using custom fallback source images.
- Integrate search/filtering components by matching user text input locally (or via mock service queries).

Phase 3: Optimistic State Management & Vouchers

Construct interactive state handlers for UI events:

- **Save to Favorites Toggle:** When clicked, update the UI color icon instantly. Send mock/real API network request. On success, retain state; on network failure, rollback UI state immediately and alert the user.
- **Voucher Ledger Panel:** Create a separate view or sidebar ledger listing recent transactions and current balance points.

Technical Assessment Questions

Q1. Rollback Strategy & Component State

Provide a clear React component code snippet demonstrating your implementation of the Favorite toggle. Walk through how you utilize the rollback logic in your catch block when mock api requests reject.

Q2. Virtualization & Large Lists Rendering

If the restaurant feed is expanded to render 200+ list cards on a mobile browser, describe how you would use rendering optimizations (like list virtualization/windowing) to maintain 60FPS scrolling.

Grading & Evaluation Rubric

Submissions are evaluated out of 100 points based on functional completeness and architecture:

Criteria	Weight	Description
UI Completeness & Aesthetics	35 pts	Clean interface layout, responsive styling, brand identity, and intuitive feedback.
Optimistic State Handlers	25 pts	Implements instantaneous toggling and robust state recovery on request errors.
Skeleton Loaders & Fallbacks	20 pts	Polished visual transition states and solid handling of missing or broken assets.
TypeScript & Reusability	20 pts	Strict props interfaces, functional component breakdown, and clean folder structures.